

Путь трансформера: от токенов к матрице

Как текст становится входом модели

Источники: С. Рашка, «Создаём большую языковую модель с нуля», гл. 2; Vaswani et al.,
arxiv.org/abs/1706.03762

Александр Чурсин
Руководитель группы
разработки в ГК «Норильский
никель»



Проверка связи

Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти

Поставьте в чат:

-  если меня видно и слышно
-  если нет

Рекомендации

→ Если смотрите с компьютера

- Используйте браузеры Google Chrome или Microsoft Edge
- Если есть проблемы с изображением или звуком, обновите страницу — **F5**

→ Если смотрите с мобильного телефона или планшета

- Перейдите с мобильного интернет-соединения на **Wi-Fi**
- Если есть проблемы с изображением или звуком, перезапустите приложение на телефоне

Правила участия

- 1 Приготовьте блокнот и ручку, чтобы записывать важные мысли и идеи
- 2 Продолжительность вебинара — 1 час 20 минут
- 3 Вы можете писать свои вопросы в чате
- 4 Запись вебинара будет доступна в личном кабинете Нетологии
- 5 Оффтоп обсуждения можно продолжить в учебном чате в ВК

Александр Чурсин

О спикере:

- TeamLead NLP в Норникеле
- Стажёр → тимлид команды из 10 за 5 лет
- Бакалавриат «Лингвистика»
- Магистр «Инженерия машинного обучения»
- Мультиагентные системы и дообучение
- Домены: мебель, туризм, медиа, металлургия



Путь из четырёх шагов

1. Токены

текст режется на части

2. Идентификаторы

каждая часть получает номер по словарю

3. Эмбеддинги

номер превращается в вектор

4. Позиции

к вектору добавляется место в тексте

Зачем это на практике

Счёт — в токенах

от токенизатора зависит цена
одного и того же текста

Предел запроса

это предел позиционного
кодирования, а не «памяти»

$\frac{1}{3}$ параметров

столько занимает входная
таблица небольшой модели

План занятия

1 Зачем текст превращать в векторы

2 Как режут текст на токены

3 Как из текста собрать батч

4 Как токен становится вектором

5 Как модель узнаёт порядок

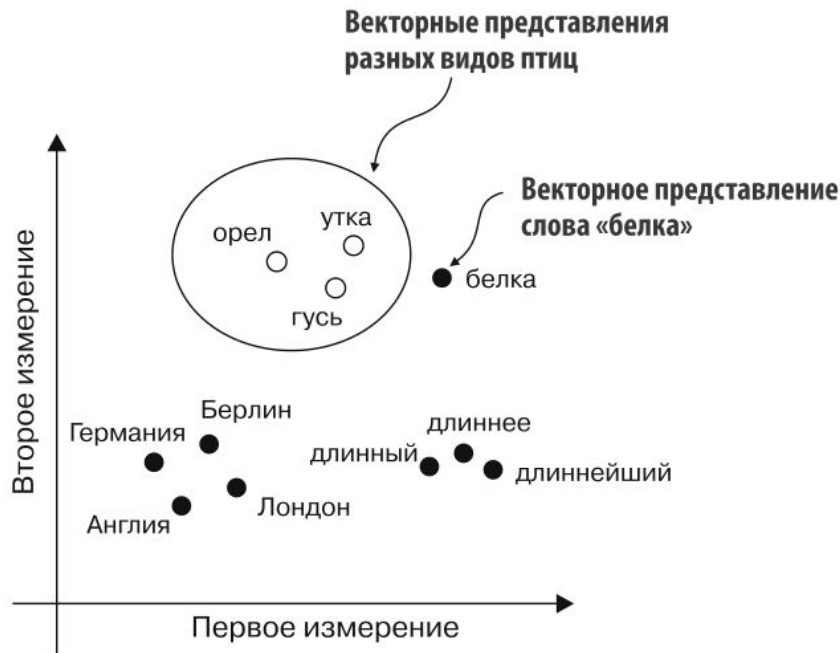
Цели занятия

- Понять, зачем текст превращают в числа
- Разобрать, как текст режут на токены и строят словарь
- Увидеть, что BPE делает с незнакомым словом
- Собрать батч скользящим окном: вход и цель со сдвигом
- Понять слой эмбединга как поиск строки в таблице
- Разобрать, откуда модель узнаёт порядок токенов

Сеть умножает числа, но не строки

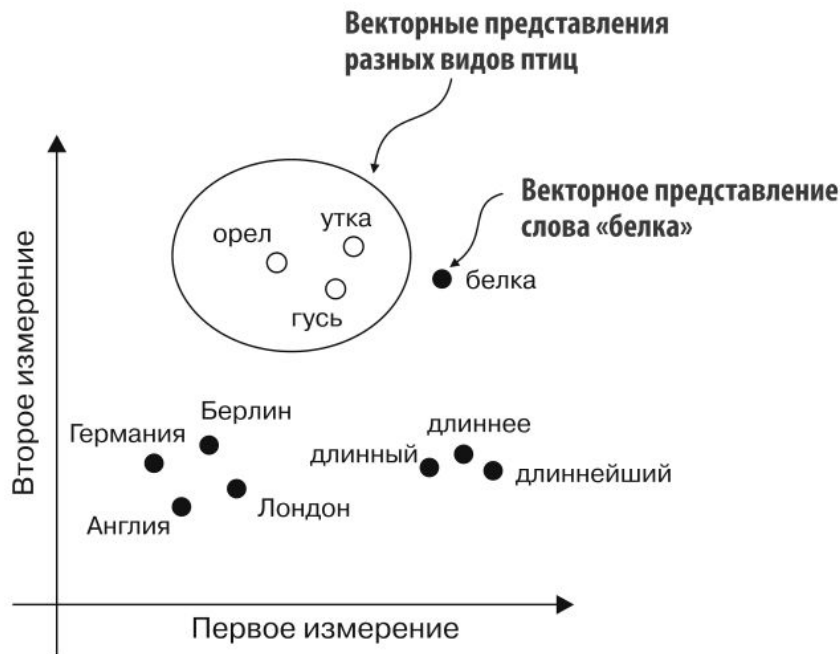
Нейронная сеть — цепочка операций над числами. Строка текста для них не аргумент.

Схожие контексты — близкие векторы



- Эмбединг — вектор из d чисел, которым описан токен

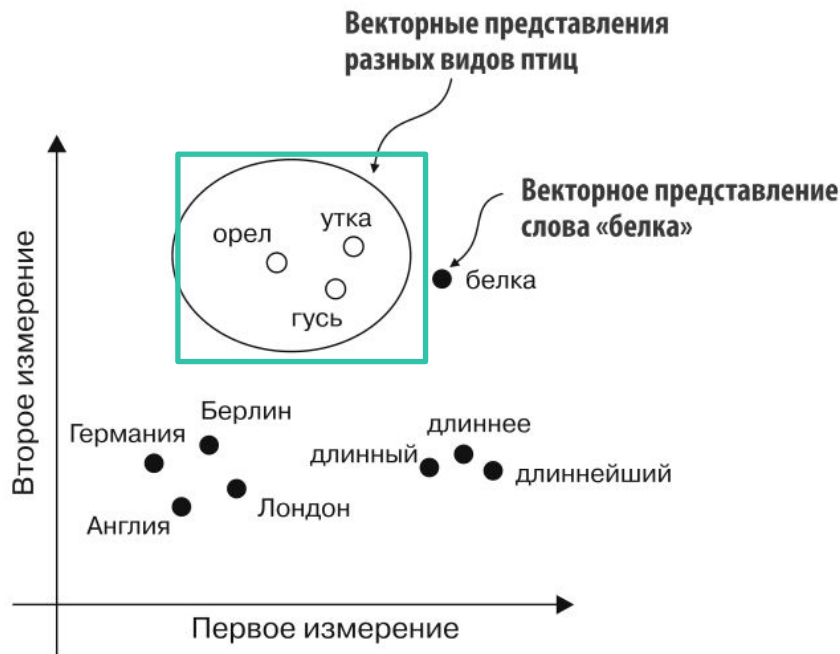
Схожие контексты — близкие векторы



- **Эмбединг** — объект как точка пространства

- Эмбединг — вектор из d чисел, которым описан токен

Схожие контексты — близкие векторы



- **Эмбединг** — объект как точка пространства
- Похожие слова оказываются рядом
- Два измерения — только чтобы нарисовать

● Эмбединг — вектор из d чисел, которым описан токен

Ключевые размерности

768

длина вектора d в GPT-2

12 288

то же в GPT-3

50 257

размер словаря V у
токенизатора GPT-2

1130

уникальных токенов в нашем
учебном тексте

Почему не взять миллион чисел на токен?

Вопрос к аудитории — 30 секунд
подумать.

**Почему не взять
миллион чисел на токен?**

Дорожает каждое умножение, а при том же корпусе лишние координаты нечем заполнить: тонких различий в данных столько нет.

План занятия

1 Зачем текст превращать в векторы

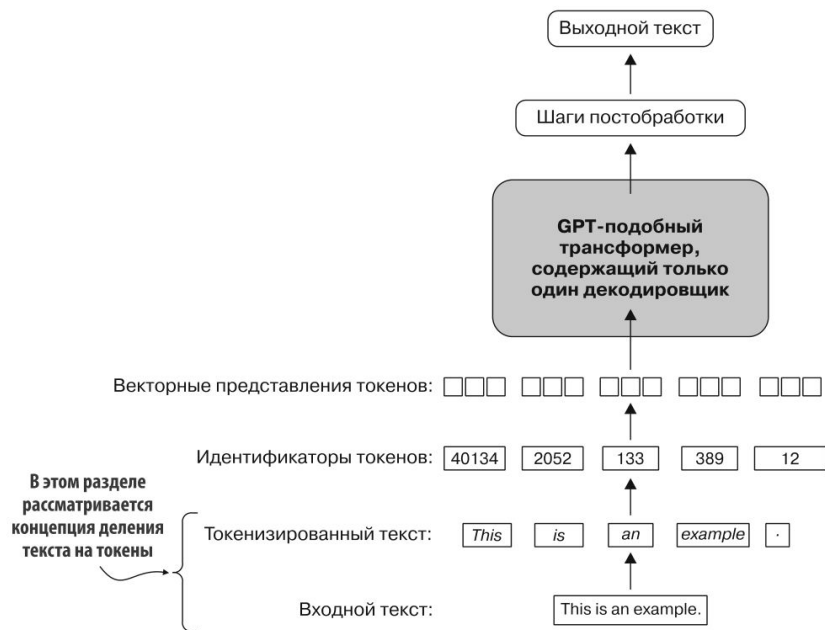
2 Как режут текст на токены

3 Как из текста собрать батч

4 Как токен становится вектором

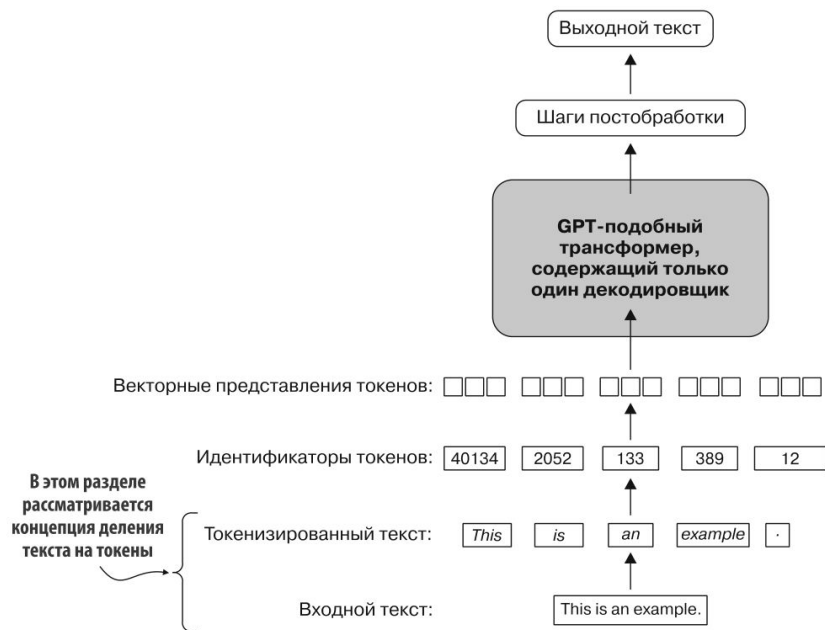
5 Как модель узнаёт порядок

Путь от текста до модели



- Эмбединг — вектор из d чисел, которым описан токен
- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре

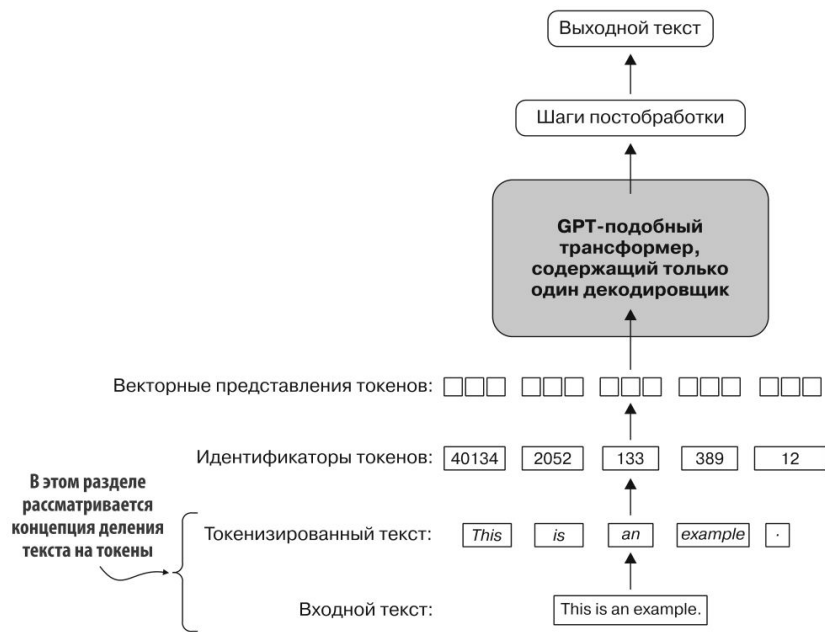
Путь от текста до модели



- Текст режется на токены

- Эмбединг — вектор из d чисел, которым описан токен
- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре

Путь от текста до модели



- Текст режется на токены
- Токены получают номера
- Номера — на векторы

- Эмбеddинг — вектор из d чисел, которым описан токен
- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре

Загружаем рассказ «The Verdict»

Один файл, 20 479 символов

```
1 import urllib.request
2
3 url = ("https://raw.githubusercontent.com/rasbt/"
4       "LLMs-from-scratch/main/ch02/01_main-chapter-code/"
5       "the-verdict.txt")
6 urllib.request.urlretrieve(url, "the-verdict.txt")
7
8 with open("the-verdict.txt", "r", encoding="utf-8") as f:
9     raw_text = f.read()
10
11 print("Total number of character:", len(raw_text)) # 20479
12 print(raw_text[:99]) # I HAD always thought Jack Gisburn rather a cheap genius--
```

Режем текст регулярным выражением

Сначала по пробелам, потом ещё по запятой и точке

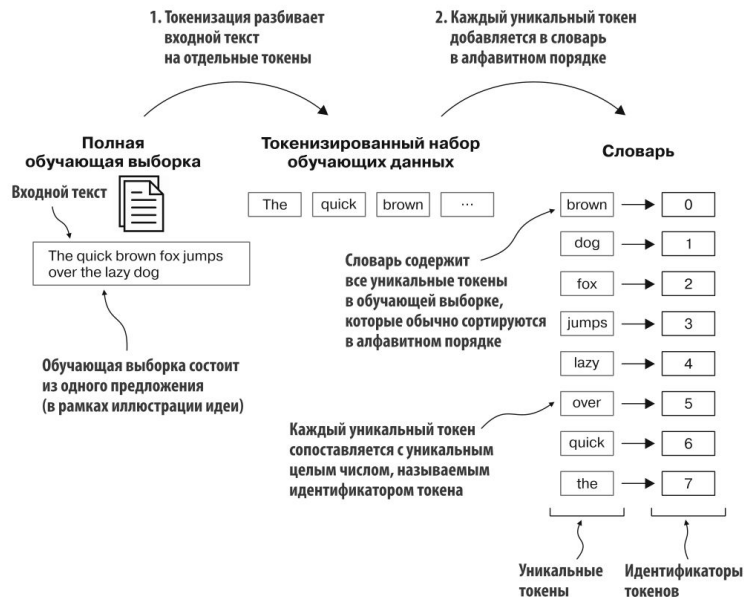
```
1 import re
2
3 text = "Hello, world. This, is a test."
4 result = re.split(r'(\s)', text)
5 print(result)
6 # ['Hello,', '', 'world.', '', 'This,', '', 'is', '', 'a', '', 'test.']
7
8 result = re.split(r'([.,])\s', text)
9 result = [item for item in result if item.strip()] # выбросить пустые и пробельные
10 print(result)
11 # ['Hello', '', 'world', '', 'This', '', 'is', 'a', 'test', '.']
```

Полный набор знаков препинания

То же выражение — на всём рассказе

```
1 | text = "Hello, world. Is this-- a test?"
2 | result = re.split(r'([.,:;?_!"()\'"]|--|\s)', text)
3 | result = [item.strip() for item in result if item.strip()]
4 | print(result)
5 | # ['Hello', ',', 'world', '.', 'Is', 'this', '--', 'a', 'test', '?']
6 |
7 | preprocessed = re.split(r'([.,:;?_!"()\'"]|--|\s)', raw_text)
8 | preprocessed = [item.strip() for item in preprocessed if item.strip()]
9 | print(len(preprocessed)) # 4690
10 | print(preprocessed[:30])
11 | # ['I', 'HAD', 'always', 'thought', 'Jack', 'Gisburn', 'rather', 'a',
12 | # 'cheap', 'genius', '--', 'though', 'a', 'good', 'fellow', 'enough', ...]
```

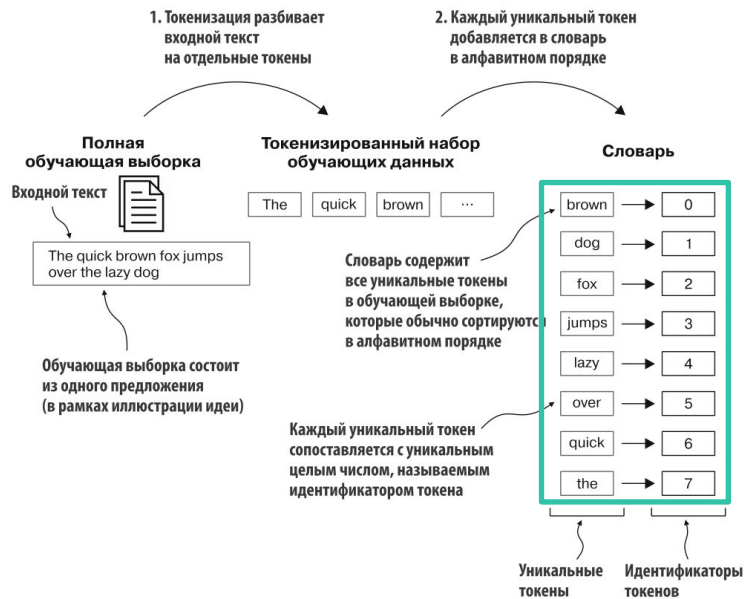
Словарь строится по обучающей выборке



- Уникальные токены — по алфавиту
- Строится один раз на весь корпус
- encode и decode — в обе стороны

- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре
- Словарь — соответствие «токен → номер», строится по обучающей выборке

Словарь строится по обучающей выборке



- Уникальные токены — по алфавиту
- Строится один раз на весь корпус
- encode и decode — в обе стороны

- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре
- Словарь — соответствие «токен → номер», строится по обучающей выборке

Строим словарь: 1130 токенов

Множество уникальных токенов, отсортированное по алфавиту

```
1 all_words = sorted(set(preprocessed))
2 vocab_size = len(all_words)
3 print(vocab_size)      # 1130
4
5 vocab = {token: integer for integer, token in enumerate(all_words)}
```

Токенизатор: encode и decode

Прямой словарь для encode, обратный — для decode

```
1 class SimpleTokenizerV1:
2     def __init__(self, vocab):
3         self.str_to_int = vocab
4         self.int_to_str = {i: s for s, i in vocab.items()}
5
6     def encode(self, text):
7         preprocessed = re.split(r'([,,:;?_!"()\'"]|--|\s)', text)
8         preprocessed = [i.strip() for i in preprocessed if i.strip()]
9         return [self.str_to_int[s] for s in preprocessed]
10
11     def decode(self, ids):
12         text = " ".join([self.int_to_str[i] for i in ids])
13         return re.sub(r'\s+([,,:;?_!"()\'"])', r'\1', text)
```

Проверяем encode и decode

Отрывок из рассказа — и слово, которого в нём нет

```
1 | tokenizer = SimpleTokenizerV1(vocab)
2 | text = """"It's the last he painted, you know,"
3 |     Mrs. Gisburn said with pardonable pride.""
4 | ids = tokenizer.encode(text)
5 | print(ids)
6 | # [1, 56, 2, 850, 988, 602, 533, 746, 5, 1126, 596, 5, 1, 67, 7, 38, 851, 1108, 754, 793, 7]
7 |
8 | print(tokenizer.decode(ids))
9 | # "' It\ ' s the last he painted, you know," Mrs. Gisburn said with pardonable pride.'
10 |
11 | text = "Hello, do you like tea?"
12 | print(tokenizer.encode(text))
13 | # KeyError: 'Hello'
```

Нет в словаре — KeyError

Так падает любой словарь, построенный по конечной выборке. Нужен способ кодировать то, чего в нём нет.

Назовите два способа починить токенизатор

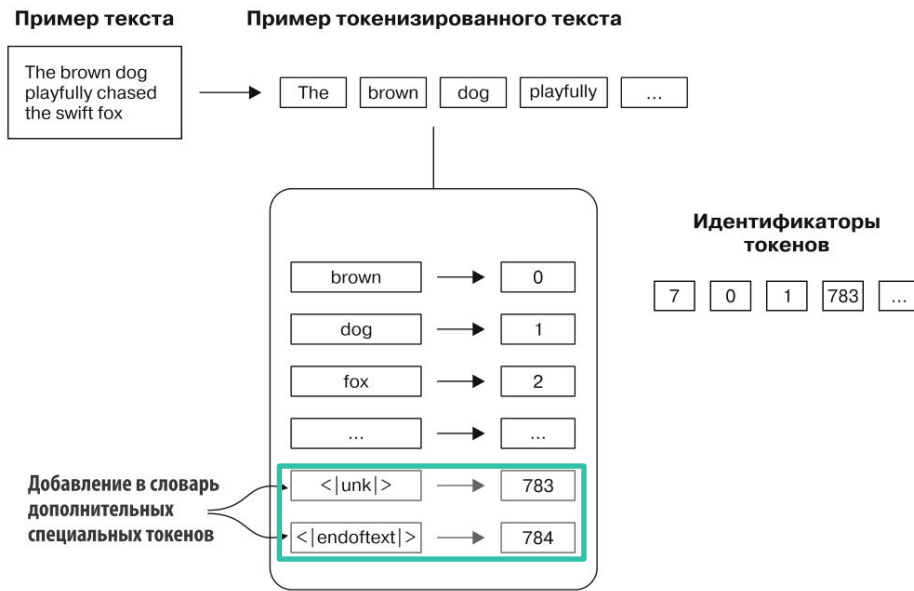
Вопрос к аудитории — 30 секунд
подумать.

Назовите два способа

починить токенизатор

Заглушка для неизвестного — с потерей слова. Разбиение на части, которые в словаре есть, — без потери. Дальше оба.

Заглушка для неизвестного слова



- unk — для слов вне словаря
- endoftext — граница источников
- Минус: слово потеряно навсегда

- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре
- Словарь — соответствие «токен → номер», строится по обучающей выборке

V2: заглушка до похода в словарь

Два новых токена в словаре и одна новая строка в encode

```
1 all_tokens = sorted(list(set(preprocessed)))
2 all_tokens.extend(["<|endoftext|>", "<|unk|>"])
3 vocab = {token: integer for integer, token in enumerate(all_tokens)} # 1132
4
5 class SimpleTokenizerV2:
6     def __init__(self, vocab):
7         self.str_to_int = vocab
8         self.int_to_str = {i: s for s, i in vocab.items()}
9
10    def encode(self, text):
11        preprocessed = re.split(r'([.,:;?_!"()\'|]|--|\s)', text)
12        preprocessed = [i.strip() for i in preprocessed if i.strip()]
13        preprocessed = [i if i in self.str_to_int else "<|unk|>" for i in preprocessed]
14        return [self.str_to_int[s] for s in preprocessed]
```

Проверяем V2 на двух предложениях

Соединяем их через endoftext

```
1 text1 = "Hello, do you like tea?"
2 text2 = "In the sunlit terraces of the palace."
3 text = " <|endoftext|> ".join((text1, text2))
4
5 tokenizer = SimpleTokenizerV2(vocab)
6 print(tokenizer.encode(text))
7 # [1131, 5, 355, 1126, 628, 975, 10, 1130, 55, 988, 956, 984, 722, 988, 1131, 7]
8
9 print(tokenizer.decode(tokenizer.encode(text)))
10 # <|unk|>, do you like tea? <|endoftext|> In the sunlit terraces of the <|unk|>.
```

Другие специальные токены

- [BOS] / [EOS] — начало и конец текста; [EOS] играет роль `<|endoftext|>`
- [PAD] — заполнитель: короткие тексты батча дополняют до длины самого длинного
- У GPT только `<|endoftext|>`: окно фиксированной длины, дополнять нечего

- Токен — слово или знак препинания, на которые режут текст
Идентификатор — номер токена в словаре
Словарь — соответствие «токен → номер», строится по обучающей выборке

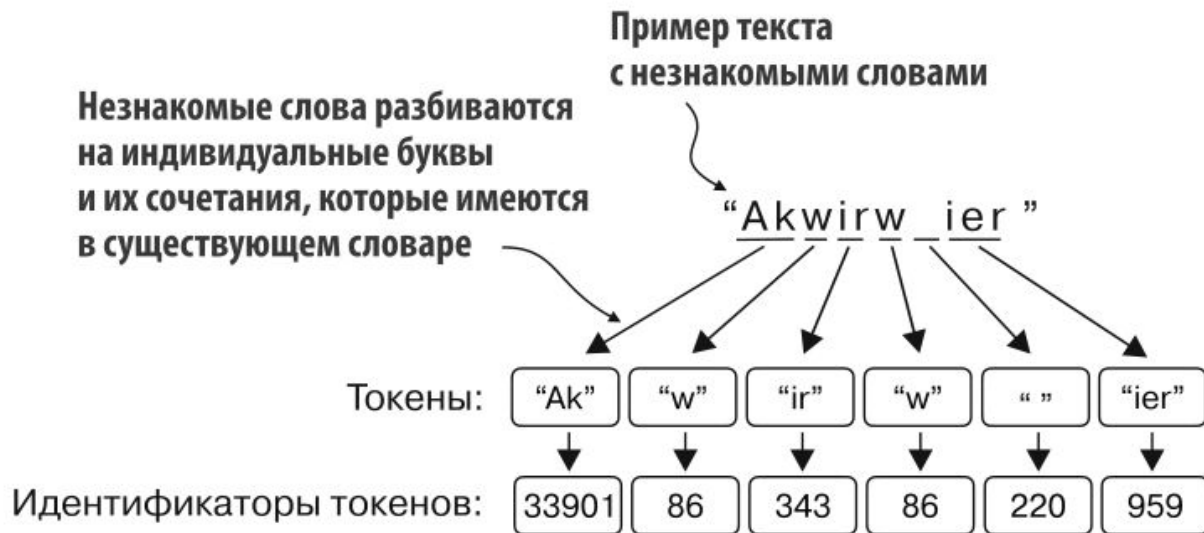
Другие специальные токены

- [BOS] / [EOS] — начало и конец текста; [EOS] играет роль `<|endoftext|>`
- [PAD] — заполнитель: короткие тексты батча дополняют до длины самого длинного
- У GPT только `<|endoftext|>`: окно фиксированной длины, дополнять нечего

Маска — запрет учитывать выбранные позиции: их вклад обнуляют перед тем, как модель сложит всё вместе

- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре
- Словарь — соответствие «токен → номер», строится по обучающей выборке

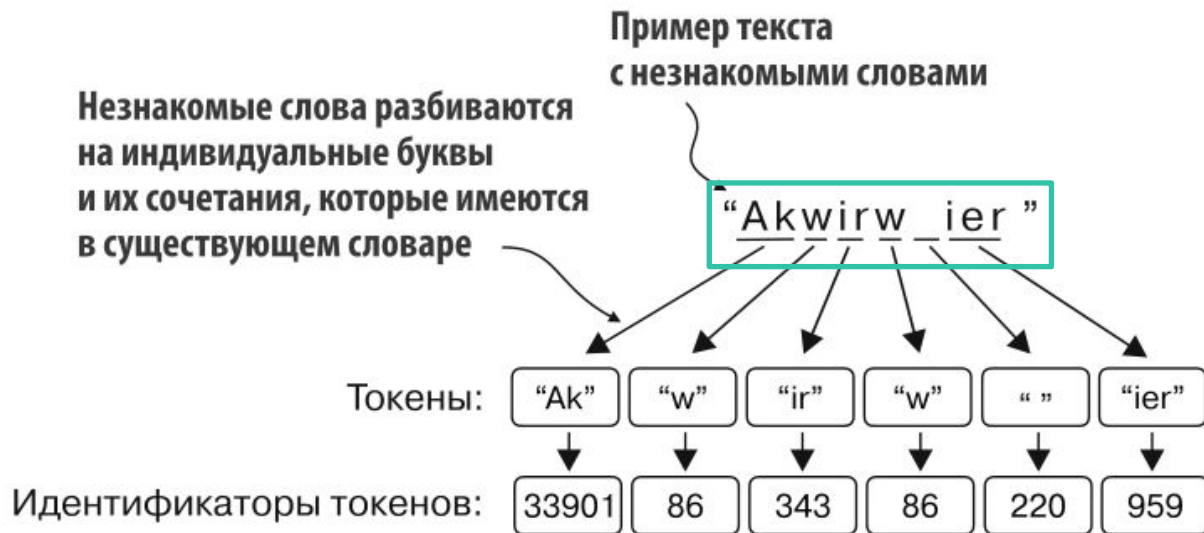
ВРЕ разбирает незнакомое слово



- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре
- Словарь — соответствие «токен → номер», строится по обучающей выборке

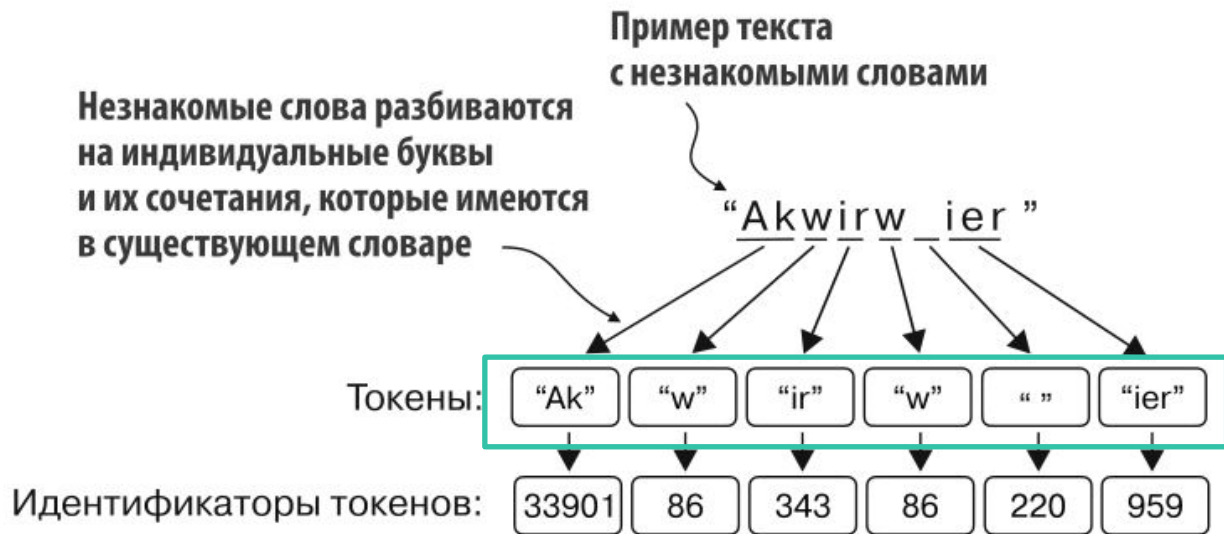
ВРЕ разбирает незнакомое слово

- Незнакомое слово
— на части словаря



- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре
- Словарь — соответствие «токен → номер», строится по обучающей выборке

ВРЕ разбирает незнакомое слово



- Незнакомое слово — на части словаря
- Старт — символы, потом частые пары
- Без заглушек: merge частых пар

- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре
- Словарь — соответствие «токен → номер», строится по обучающей выборке

Частота — это сжатие

Склеили самую частую пару — сильнее всего укоротили корпус на одну потраченную ячейку словаря. Число слияний задаёт инженер.

Три слияния: корпус из пяти слов

Шаг	Самая частая пара	Частота	Новая единица (merge)
0	—	—	b, g, h, n, p, s, u
1	u, g	20	ug
2	u, n	16	un
3	h, ug	15	hug

Функция слияния merge

Корпус из пяти слов с частотами — и десять строк ядра алгоритма

```
1 corpus = {"hug": 10, "pug": 5, "pun": 12, "bun": 4, "hugs": 5}
2 splits = {word: list(word) for word in corpus} # старт: слово -> список символов
3
4 def merge(indices, pair, new_index):
5     new_indices, i = [], 0
6     while i < len(indices):
7         if i + 1 < len(indices) and indices[i] == pair[0] and indices[i+1] == pair[1]:
8             new_indices.append(new_index); i += 2
9         else:
10             new_indices.append(indices[i]); i += 1
11     return new_indices
```

Считаем частоты соседних пар

Пара весит столько, сколько раз встречается слово

```
1 from collections import Counter
2
3 def count_pairs(splits, corpus):
4     pairs = Counter()
5     for word, freq in corpus.items():
6         symbols = splits[word]
7         for a, b in zip(symbols, symbols[1:]):
8             pairs[(a, b)] += freq
9     return pairs
```

Цикл обучения: три слияния

Посчитать пары → взять самую частую → применить ко всем словам

```
1 merges = []                # выученные правила -- в порядке обучения
2 for step in range(3):      # число слияний задаёт размер словаря
3     pairs = count_pairs(splits, corpus)
4     best = max(pairs.items(), key=lambda kv: (kv[1], kv[0]))[0]
5     new_token = best[0] + best[1]
6     merges.append(best)
7     splits = {w: merge(s, best, new_token) for w, s in splits.items()}
8     print(step + 1, best, "->", new_token, pairs[best])
9     # 1 ('u', 'g') -> ug 20
10    # 2 ('u', 'n') -> un 16
11    # 3 ('h', 'ug') -> hug 15
```

Кодируем новое слово по списку правил

Частоты больше не считаются — правила применяются в порядке обучения

```
1 vocab = sorted(set(''.join(corpus))) + [a + b for a, b in merges]
2 print(vocab)    # ['b', 'g', 'h', 'n', 'p', 's', 'u', 'ug', 'un', 'hug']
3
4 def encode(word, merges):
5     symbols = list(word)
6     for pair in merges:
7         symbols = merge(symbols, pair, pair[0] + pair[1])
8     return symbols
9
10 for word in ["hug", "pug", "pun", "bun", "hugs", "bug"]:
11     print(word, "->", encode(word, merges))
12 # hug -> ['hug']   pug -> ['p', 'ug']   pun -> ['p', 'un']
13 # bun -> ['b', 'un']   hugs -> ['hug', 's']   bug -> ['b', 'ug']
```

Готовый ВРЕ: tiktoken

Тот же интерфейс encode и decode, что у нашего токенизатора

```
1 import tiktoken
2
3 tokenizer = tiktoken.get_encoding("gpt2")
4
5 text = (
6     "Hello, do you like tea? <|endoftext|> In the sunlit terraces"
7     " of someunknownPlace."
8 )
9 integers = tokenizer.encode(text, allowed_special={"<|endoftext|>"})
10 print(integers)
11 # [15496, 11, 466, 345, 588, 8887, 30, 220, 50256, 554, 262, 4252, 18250,
12 #  8812, 2114, 286, 617, 34680, 27271, 13]
13
14 print(tokenizer.decode(integers)) # ...terraces of someunknownPlace.
```

Три вещи о токенизаторе

Токен — это не слово

Hello — один токен,
someunknownPlace — три,
Akwirw ier — шесть

Пробел прилипает

«do» в начале строки и «_do»
в середине — разные единицы
словаря

Числа режутся группами цифр

«1024» и «1025» распадаются
по-разному — отсюда
трудности с арифметикой

Коэффициент сжатия: байт на токен

Схема	Байт на токен	Словарь	Неизвестные слова
Посимвольная	≈ 1	сотни тысяч символов	нет
Пословная	лучший	неограничен	есть
BPE	≈ 4	фиксирован, 50 257	нет

Русский текст вдвое-втрое дороже

Токенизатор GPT-2 учился на
английском и режет русский мельче.
Бьёт по счёту и по контексту.

Что уже собрано

- Токен — слово, часть слова или знак
- Словарь: токен → идентификатор, и обратно
- Заглушка теряет слово, ВРЕ — нет
- Сравниваем по байтам на токен

Совсем без токенизатора?

ByT5, 2021

модель прямо на байтах:
устойчивее к опечаткам, но
последовательности в разы
длиннее

MegaByte, BLT

байты группируют в патчи и
цену длины снижают

Стандарт сегодня

BPE и его варианты

План занятия

1 Зачем текст превращать в векторы

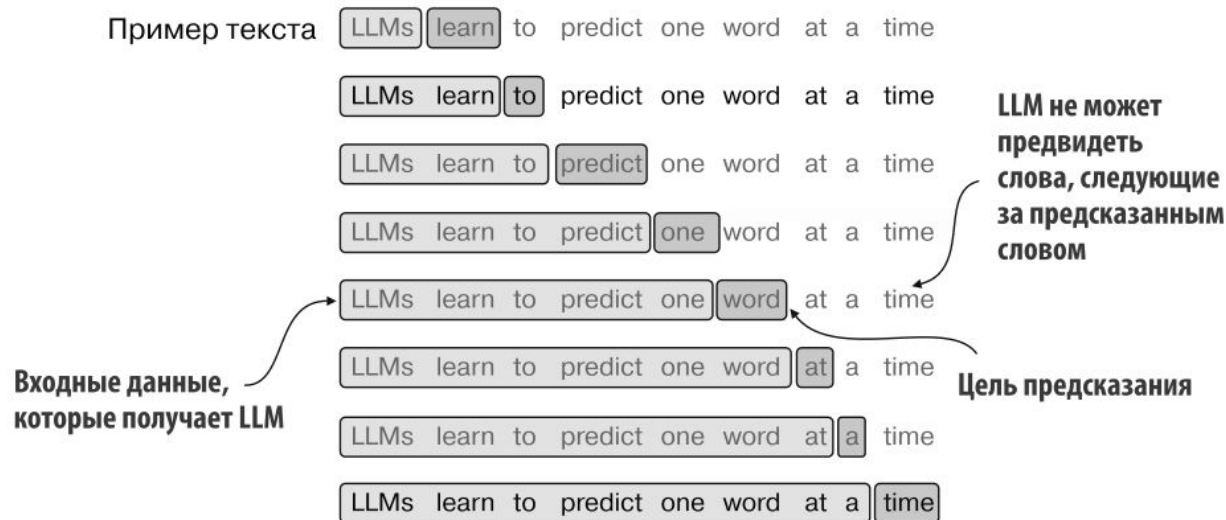
2 Как режут текст на токены

3 Как из текста собрать батч

4 Как токен становится вектором

5 Как модель узнаёт порядок

Модель предсказывает следующий токен



- Вход — начало текста
- Цель — один токен следом
- Из фразы столько задач, сколько слов

- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре
- Словарь — соответствие «токен → номер», строится по обучающей выборке

Цель — вход, сдвинутый на токен

Одна и та же лента, два среза

```
1 with open("the-verdict.txt", "r", encoding="utf-8") as f:
2     raw_text = f.read()
3
4     enc_text = tokenizer.encode(raw_text)
5     print(len(enc_text))    # 5145 -- токенов больше, чем у пословного: BPE дробит редкие слова
6
7     enc_sample = enc_text[50:] # отступаем 50 токенов: дальше фрагмент нагляднее
8
9     context_size = 4          # сколько токенов модель видит на входе
10    x = enc_sample[:context_size]
11    y = enc_sample[1:context_size + 1]
12    print(f"x: {x}")          # x: [290, 4920, 2241, 287]
13    print(f"y: {y}")          # y:  [4920, 2241, 287, 257]
```

Каждый префикс — одна задача

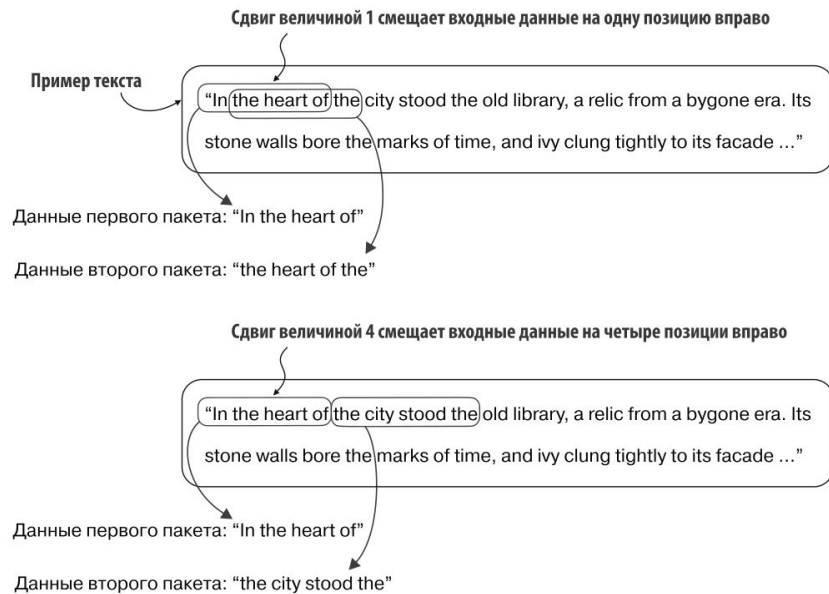
Цикл по длине контекста

```
1 for i in range(1, context_size + 1):
2     context = enc_sample[:i]
3     desired = enc_sample[i]
4     print(tokenizer.decode(context), "---->", tokenizer.decode([desired]))
5
6 # and ----> established
7 # and established ----> himself
8 # and established himself ----> in
9 # and established himself in ----> a
```

Минимизируется перекрёстная энтропия

Между распределением по словарю и настоящим следующим токеном. Через неё градиент доходит до таблицы эмбедингов.

Sliding window: окно и шаг



- `max_length` — длина окна
- `stride` — шаг окна по тексту
- Шаг = длине окна:
ОКНА ВСТЫК

- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре
- Словарь — соответствие «токен → номер», строится по обучающей выборке

Sliding window режет текст на примеры

`max_length` — длина окна, `stride` — шаг

```
1 import torch
2 from torch.utils.data import Dataset, DataLoader
3
4 class GPTDatasetV1(Dataset):
5     def __init__(self, txt, tokenizer, max_length, stride):
6         self.input_ids, self.target_ids = [], []
7         token_ids = tokenizer.encode(txt)
8
9         for i in range(0, len(token_ids) - max_length, stride):
10             self.input_ids.append(torch.tensor(token_ids[i:i + max_length]))
11             self.target_ids.append(torch.tensor(token_ids[i + 1: i + max_length + 1]))
12
13     def __len__(self):
14         return len(self.input_ids)
15
16     def __getitem__(self, i):
17         return self.input_ids[i], self.target_ids[i]
```

DataLoader собирает примеры в батч

Обёртка над датасетом с настройками загрузки

```
1 def create_dataloader_v1(txt, batch_size=4, max_length=256,  
2     stride=128, shuffle=True, drop_last=True,  
3     num_workers=0):  
4     tokenizer = tiktoken.get_encoding("gpt2")  
5     dataset = GPTDatasetV1(txt, tokenizer, max_length, stride)  
6     dataloader = DataLoader(  
7         dataset,  
8         batch_size=batch_size,  
9         shuffle=shuffle,  
10        drop_last=drop_last,    # отбросить неполный последний батч  
11        num_workers=num_workers  
12    )  
13    return dataloader
```

Батч размера 1: механика сдвига

stride = 1 — окна перекрываются

```
1 | dataloader = create_dataloader_v1(  
2 |     raw_text, batch_size=1, max_length=4, stride=1, shuffle=False)  
3 |  
4 | data_iter = iter(dataloader)  
5 | print(next(data_iter))  
6 | # [tensor([[ 40, 367, 2885, 1464]]), tensor([[ 367, 2885, 1464, 1807]])]  
7 | print(next(data_iter))  
8 | # [tensor([[ 367, 2885, 1464, 1807]]), tensor([[2885, 1464, 1807, 3619]])]
```

Настройки загрузчика

drop_last

неполный последний батч
отбрасывают: иначе в конце
эпохи на графике выброс

Размер батча

меньше батч — меньше
памяти, но шумнее градиент

Размер окна

здесь учебный; у настоящих
LLM не меньше 256

Батч 8 × 4: вход и цель

stride = max_length — окна встык

```
1 | dataloader = create_dataloader_v1(  
2 |     raw_text, batch_size=8, max_length=4, stride=4, shuffle=False)  
3 |  
4 | inputs, targets = next(iter(dataloader))  
5 | print("Inputs:\n", inputs)    # тензор 8 x 4: батч из восьми примеров  
6 | print("\nTargets:\n", targets) # тот же тензор, сдвинутый на один токен
```

План занятия

1 Зачем текст превращать в векторы

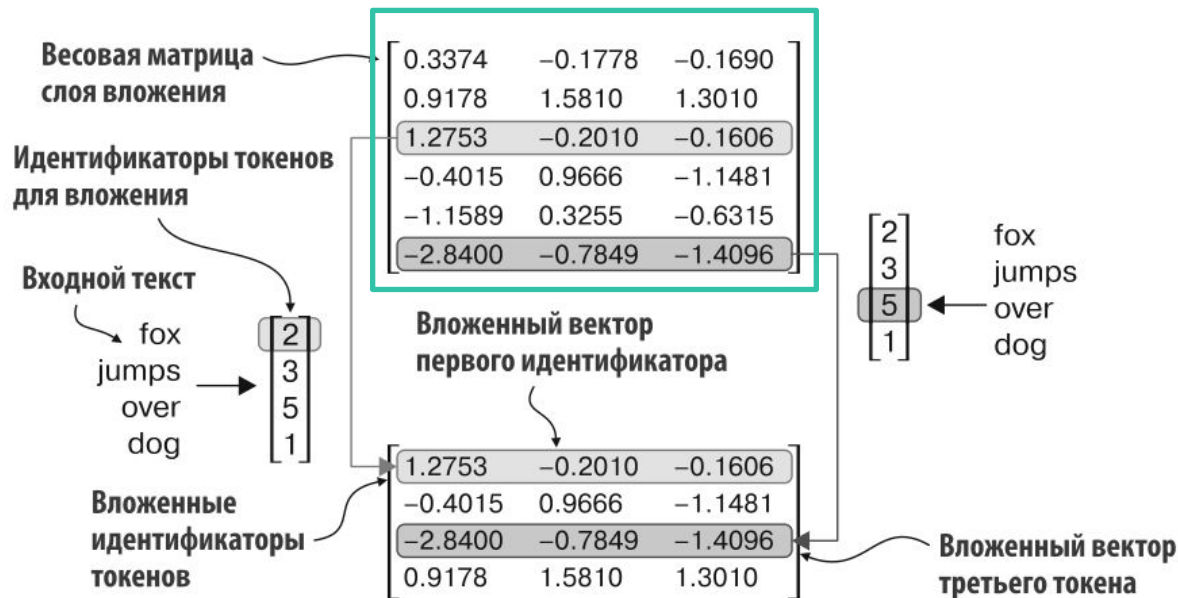
2 Как режут текст на токены

3 Как из текста собрать батч

4 Как токен становится вектором

5 Как модель узнаёт порядок

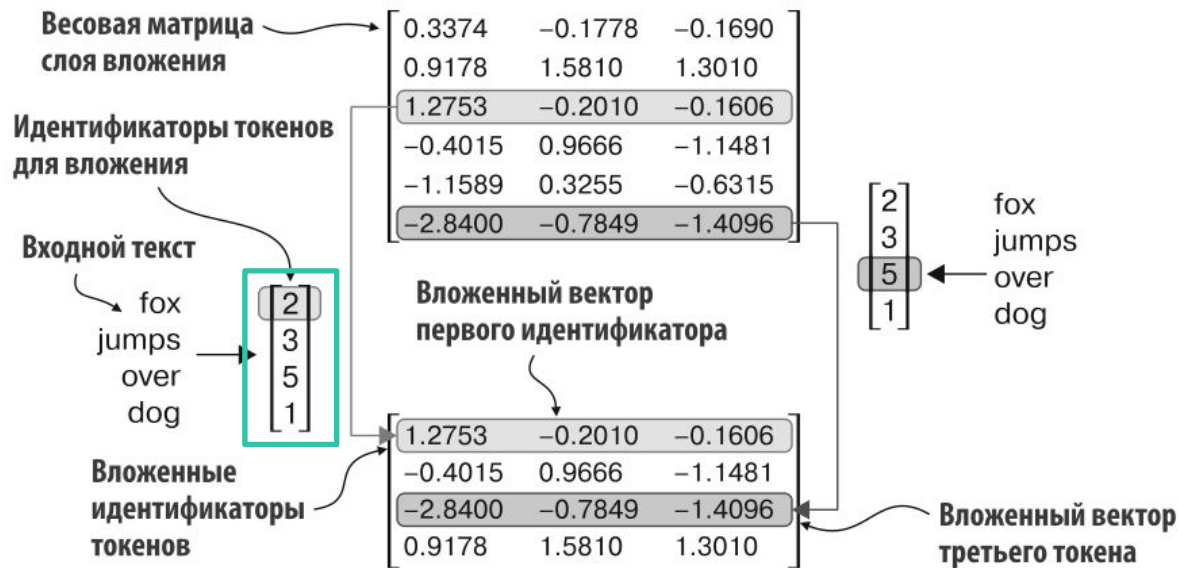
Слой эмбединга — поиск строки



- Инициализация: случайные строки

- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре
- Словарь — соответствие «токен → номер», строится по обучающей выборке

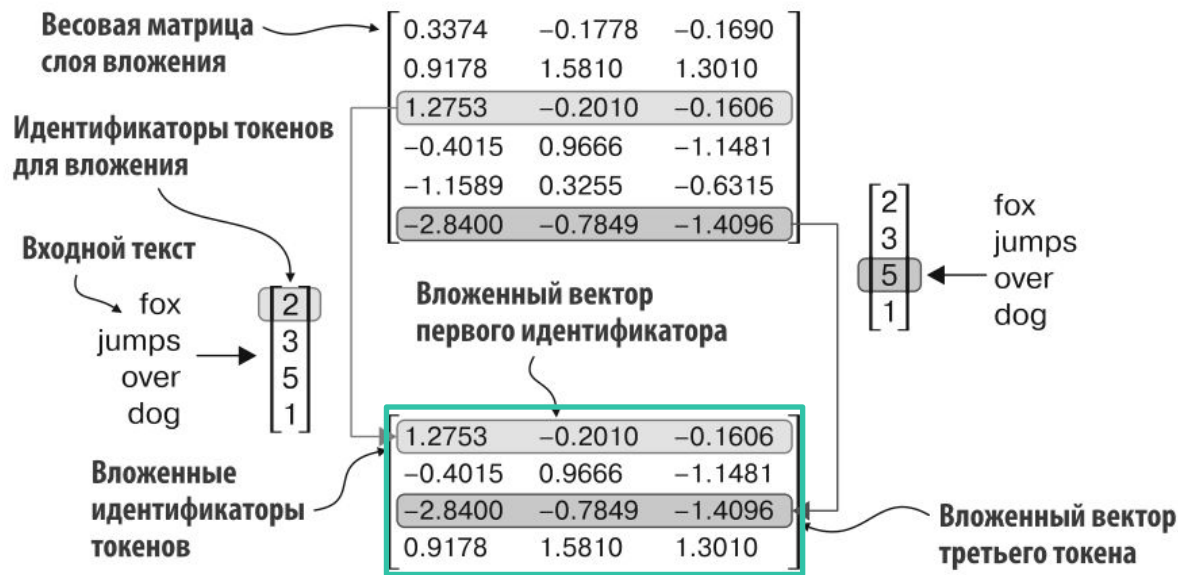
Слой эмбединга — поиск строки



- Инициализация: случайные строки
- Идентификатор выбирает строку

- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре
- Словарь — соответствие «токен → номер», строится по обучающей выборке

Слой эмбединга — поиск строки



- Инициализация: случайные строки
- Идентификатор выбирает строку
- Она и становится строкой выхода

- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре
- Словарь — соответствие «токен → номер», строится по обучающей выборке

Таблица эмбеддингов — обычный слой

Заполняется случайными числами, обучается вместе с моделью

```
1 input_ids = torch.tensor([2, 3, 5, 1])
2 vocab_size, output_dim = 6, 3
3
4 torch.manual_seed(123)
5 embedding_layer = torch.nn.Embedding(vocab_size, output_dim)
6 print(embedding_layer.weight)
7 # tensor([[[ 0.3374, -0.1778, -0.1690],
8 #          [ 0.9178,  1.5810,  1.3010],
9 #          [ 1.2753, -0.2010, -0.1606],
10 #          [-0.4015,  0.9666, -1.1481],
11 #          [-1.1589,  0.3255, -0.6315],
12 #          [-2.8400, -0.7849, -1.4096]], requires_grad=True)
13 print(embedding_layer(torch.tensor([3])))
14 # tensor([[-0.4015,  0.9666, -1.1481]], grad_fn=<EmbeddingBackward0>)
```

One-hot: единица выбирает строку

one-hot идентификатора 3 (1 × 6) × весовая матрица (6 × 3) = строка 3 (1 × 3)

0	0	0	1	0	0
---	---	---	---	---	---

0	0.3374	-0.1778	-0.1690
1	0.9178	1.5810	1.3010
2	1.2753	-0.2010	-0.1606
3	-0.4015	0.9666	-1.1481
4	-1.1589	0.3255	-0.6315
5	-2.8400	-0.7849	-1.4096

-0.4015	0.9666	-1.1481
---------	--------	---------

- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре
- Словарь — соответствие «токен → номер», строится по обучающей выборке

One-hot: единица выбирает строку

one-hot идентификатора 3 (1 × 6) × весовая матрица (6 × 3) = строка 3 (1 × 3)

0	0	0	1	0	0
---	---	---	----------	---	---

0	0.3374	-0.1778	-0.1690
1	0.9178	1.5810	1.3010
2	1.2753	-0.2010	-0.1606
3	-0.4015	0.9666	-1.1481
4	-1.1589	0.3255	-0.6315
5	-2.8400	-0.7849	-1.4096

-0.4015	0.9666	-1.1481
----------------	---------------	----------------

- One-hot: нули и одна единица

- Эквивалентно умножению на веса

- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре
- Словарь — соответствие «токен → номер», строится по обучающей выборке

Тот же результат: one-hot × веса

Поиск строки — это умножение one-hot на матрицу весов

```
1 | onehot = torch.nn.functional.one_hot(input_ids, num_classes=vocab_size).float()
2 | manual = onehot @ embedding_layer.weight      # умножение one-hot на веса
3 | lookup = embedding_layer(input_ids)           # то же самое индексацией
4 | print(torch.allclose(manual, lookup))          # True
```

План занятия

1 Зачем текст превращать в векторы

2 Как режут текст на токены

3 Как из текста собрать батч

4 Как токен становится вектором

5 Как модель узнаёт порядок

Порядок в эмбедингах не закодирован

- Слой эмбединга: один идентификатор — одна строка
- Где бы токен ни стоял, вектор один и тот же
- self-attention сравнивает все токены со всеми — порядок тоже не учитывает

● Токен — слово или знак препинания, на которые режут текст
Идентификатор — номер токена в словаре
Словарь — соответствие «токен → номер», строится по обучающей выборке

Порядок в эмбедингах не закодирован

- Слой эмбединга: один идентификатор — одна строка
- Где бы токен ни стоял, вектор один и тот же
- self-attention сравнивает все токены со всеми — порядок тоже не учитывает

Позицию приходится сообщать отдельно. Проверьте сами: один и тот же идентификатор на первой и третьей позиции даёт одну и ту же строку

- Токен — слово или знак препинания, на которые режут текст
Идентификатор — номер токена в словаре
Словарь — соответствие «токен → номер», строится по обучающей выборке

Вход модели — сумма двух эмбеддингов



- Эмбеддинг токена — что за токен
- Эмбеддинг позиции — где он стоит
- Сумма, не конкатенация

- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре
- Словарь — соответствие «токен → номер», строится по обучающей выборке

Сложить или приписать сбоку

Конкатенация

приписать вектор позиции
сбоку: $d = 256 \rightarrow 512$, и все
матрицы модели растут вслед

Сложение

d остаётся 256; позиция
ложится в те же координаты

Почему сложение не портит смысл

- Почти ортогональны — почти перпендикулярны: сдвиг вдоль одного направления почти не меняет положение вдоль другого
- В 256 координатах два случайных направления почти не мешают друг другу
- Смысл и позицию модель разносит по разным направлениям; разделение неидеально — часть ёмкости уходит на позицию

- Токен — слово или знак препинания, на которые режут текст
Идентификатор — номер токена в словаре
Словарь — соответствие «токен → номер», строится по обучающей выборке

Почему сложение не портит смысл

- Почти ортогональны — почти перпендикулярны: сдвиг вдоль одного направления почти не меняет положение вдоль другого
- В 256 координатах два случайных направления почти не мешают друг другу
- Смысл и позицию модель разносит по разным направлениям; разделение неидеально — часть ёмкости уходит на позицию

GPT: абсолютные позиционные эмбединги обучаются — ещё одна таблица nn.Embedding, строка на позицию

- Токен — слово или знак препинания, на которые режут текст
Идентификатор — номер токена в словаре
Словарь — соответствие «токен → номер», строится по обучающей выборке

Позиции — ещё одна таблица

Строк столько, сколько мест в контекстном окне

```
1 vocab_size = 50257
2 output_dim = 256
3 token_embedding_layer = torch.nn.Embedding(vocab_size, output_dim)
4
5 max_length = 4
6 dataloader = create_dataloader_v1(raw_text, batch_size=8, max_length=max_length,
7                                   stride=max_length, shuffle=False)
8 inputs, targets = next(iter(dataloader))
9 token_embeddings = token_embedding_layer(inputs)      # torch.Size([8, 4, 256])
10
11 context_length = max_length
12 pos_embedding_layer = torch.nn.Embedding(context_length, output_dim)
13 pos_embeddings = pos_embedding_layer(torch.arange(context_length)) # [4, 256]
14 input_embeddings = token_embeddings + pos_embeddings  # torch.Size([8, 4, 256])
```

Интерполяция — достроить строки

Таблицу на 1024 строки растягивают на 2048: позиция 1000 читает середину между старыми 500 и 501, затем модель дообучают.

4 × 256

Матрица входа собрана: строка — токен, столбец
— координата; в каждой строке и смысл, и
позиция

Таблице нужна строка на каждую позицию

Оригинальный трансформер позицию не хранит, а вычисляет по формуле:
параметров нет, позиция любая.

Формула вместо таблицы

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

- Пара координат — $\sin$ и $\cos$ одного угла
- Угол = позиция / знаменатель пары
- Первые пары быстрые, последние медленные

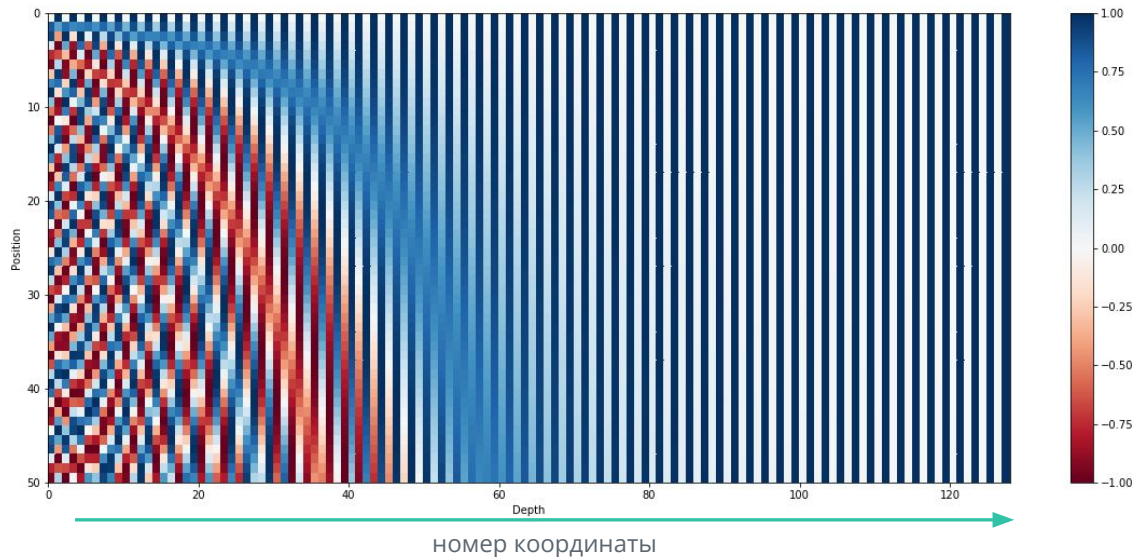
- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре
- Словарь — соответствие «токен → номер», строится по обучающей выборке

Синусоидальная функция позиций

Замена обучаемой таблице — в одну функцию

```
1 def sinusoidal_pe(context_length, output_dim):
2     pos = torch.arange(context_length, dtype=torch.float32).unsqueeze(1)
3     i = torch.arange(0, output_dim, 2, dtype=torch.float32) # номера чётных координат
4     angles = pos / (10000 ** (i / output_dim)) # у каждой пары своя частота
5     pe = torch.zeros(context_length, output_dim)
6     pe[:, 0::2] = torch.sin(angles) # чётные координаты -- синус
7     pe[:, 1::2] = torch.cos(angles) # нечётные -- косинус
8     return pe
9
10 pe = sinusoidal_pe(context_length, output_dim)
11 print(pe.shape) # torch.Size([4, 256])
12
13 input_embeddings = token_embeddings + pe # вместо pos_embedding_layer
```

Быстрые и медленные координаты



- Строка — вектор одной позиции
- Слева частая рябь: различает соседей
- Справа медленно: начало или конец

● Токен — слово или знак препинания, на которые режут текст
Идентификатор — номер токена в словаре
Словарь — соответствие «токен → номер», строится по обучающей выборке

Источник: A. Kazemnejad, «Transformer Architecture: The Positional Encoding»

Контекст 1024, пришёл текст на 2000

Что будет с таблицей и что с формулой?

**Контекст 1024,
пришёл текст на 2000**

Таблица: строк с 1025-й нет — текст
режут или модель падает. Формула
посчитает любую позицию
(экстраполяция), но качество падает.

Определена везде, работает не везде

- Экстраполяция — работа на длинах больше обученных
- За пределами обученной длины качество падает быстро
- Практическую экстраполяцию решают внутри внимания, а не на входе

● Токен — слово или знак препинания, на которые режут текст
Идентификатор — номер токена в словаре
Словарь — соответствие «токен → номер», строится по обучающей выборке

Определена везде, работает не везде

- Экстраполяция — работа на длинах больше обученных
- За пределами обученной длины качество падает быстро
- Практическую экстраполяцию решают внутри внимания, а не на входе

GPT взял обучаемую таблицу: длина контекста фиксирована архитектурой, а таблица проще

- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре
- Словарь — соответствие «токен → номер», строится по обучающей выборке

Определена везде, работает не везде

- Экстраполяция — работа на длинах больше обученных
- За пределами обученной длины качество падает быстро
- Практическую экстраполяцию решают внутри внимания, а не на входе

GPT взял обучаемую таблицу: длина контекста фиксирована архитектурой, а таблица проще

Обе схемы абсолютные: «я токен номер восемь», а не «мы в двух шагах друг от друга» — относительные разберём с вниманием

- Токен — слово или знак препинания, на которые режут текст
- Идентификатор — номер токена в словаре
- Словарь — соответствие «токен → номер», строится по обучающей выборке

Откуда взялась формула

Двоичные разряды

младшие переключаются часто, старшие редко — синусоиды делают то же плавно

Сдвиг = поворот

сдвиг позиции на k поворачивает пару $\sin$ - $\cos$ на один и тот же угол

Где читать

заметка Kazemnejad и разбор в блоге HuggingFace — ссылки в конце

Что мы собрали

- Токенизатор: encode и decode
- BPE обходится без заглушек
- Sliding window собирает батчи
- Слой эмбединга — поиск строки
- Позиция прибавляется отдельным вектором

Итоги занятия

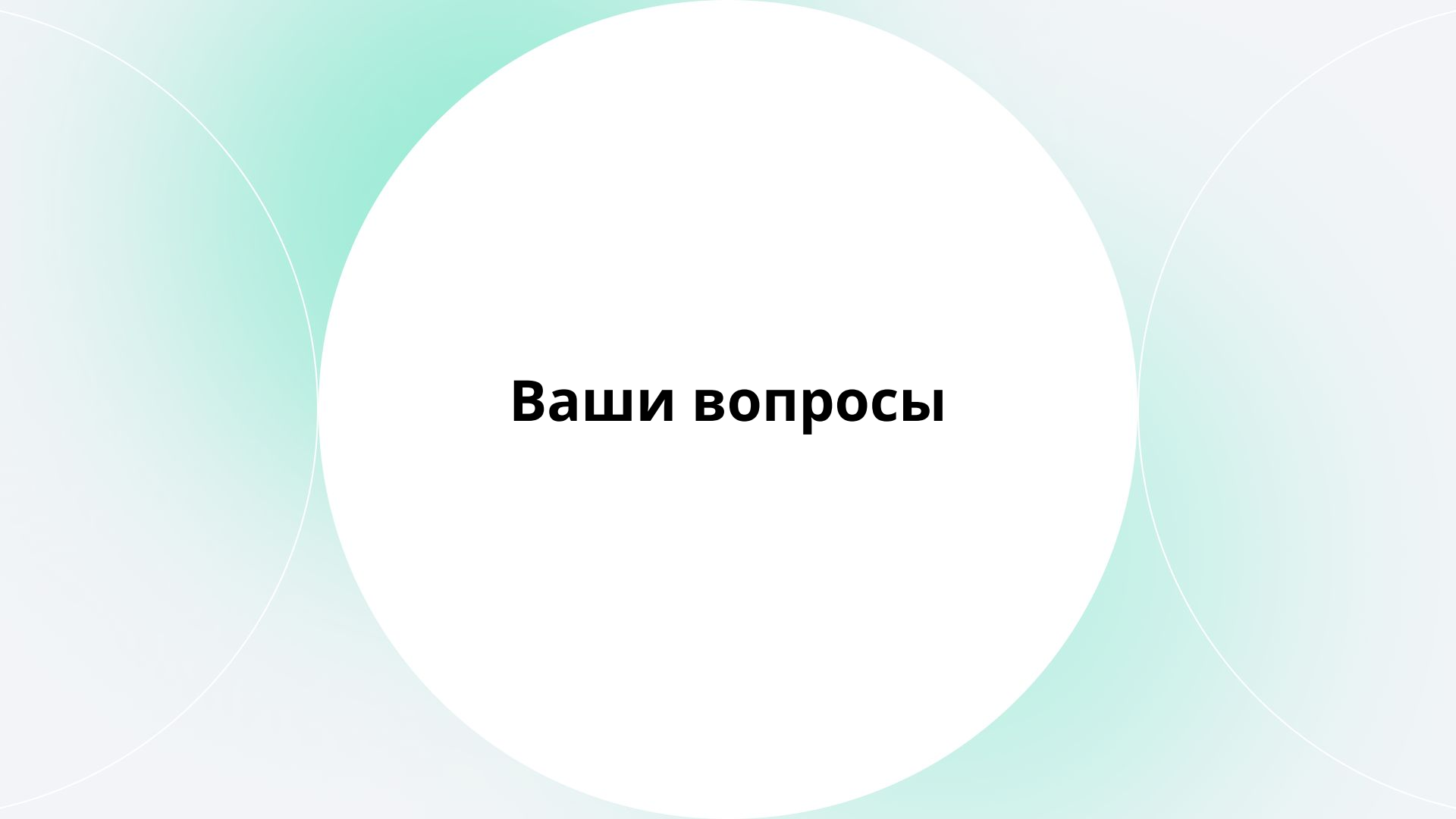
- Токен — слово, часть слова или знак; словарь даёт номер
- Заглушка теряет слово, BPE — нет: собирает из кусочков
- Сравниваем по байтам на токен: русский дороже втрое
- Sliding window даёт пары «вход — цель со сдвигом»
- Эмбеддинг — поиск строки, он же умножение на one-hot
- Позиция прибавляется вектором: таблицей или формулой

Токенизатор — часть контракта модели

Фиксируется до обучения и заменить его
нельзя: всё, что он разрезал неудачно,
модель наследует навсегда.

Что читать дальше

- Attention Is All You Need: arxiv.org/abs/1706.03762
- ByT5, модель без токенизатора: arxiv.org/abs/2105.13626
- Byte Latent Transformer: arxiv.org/abs/2412.09871
- Kazemnejad, The Positional Encoding — kazemnejad.com/blog



Ваши вопросы

Путь трансформера: от токенов к матрице

Как текст становится входом модели

Источники: С. Рашка, «Создаём большую языковую модель с нуля», гл. 2; Vaswani et al.,
arxiv.org/abs/1706.03762

Александр Чурсин
Руководитель группы
разработки в ГК «Норильский
никель»

